

Mini-Issue LAB 2

目标

实现 Issue CRUD 系统。

功能：

- 创建 Issue
- 查询 Issue 列表
- 查询单个 Issue
- 更新 Issue
- 删除 Issue

技术栈：

- Gin
- sqlx
- MySQL
- JWT

本 LAB 默认：

LAB1 已完成

即：

- 用户系统
- JWT middleware
- MySQL
- 分层架构

已经存在。

一、本 LAB 核心目标

本 LAB 重点不是 CRUD。

重点是：

资源权限控制

即：

谁创建
谁修改
谁删除

每个 issue 必须属于某个用户。

二、数据库设计

创建表：

issues

issues 字段

字段	类型	说明
id	bigint	主键
user_id	bigint	创建者
title	varchar(255)	标题
content	text	内容
status	varchar(32)	状态
priority	int	优先级
created_at	timestamp	创建时间
updated_at	timestamp	更新时间

约束

要求：

id 自增主键
user_id 非空
title 非空
status 非空
priority 非空

三、状态设计

允许：

状态
OPEN
IN_PROGRESS
DONE

非法状态必须报错。

四、优先级设计

允许：

priority	含义
1	LOW
2	MEDIUM
3	HIGH

非法 priority 必须报错。

五、新增目录与文件

新增：

```
internal/model/issue.go
internal/dao/issue_dao.go
internal/service/issue_service.go
internal/controller/issue_controller.go
```

六、整体架构

调用链：

```
HTTP Request
↓
```

```
Router
↓
Controller
↓
Service
↓
DAO
↓
MySQL
```

七、模型设计

文件：

```
internal/model/issue.go
```

需要定义的数据结构

Issue

数据库模型。

字段至少包含：

字段

ID

UserID

Title

Content

Status

Priority

CreatedAt

UpdatedAt

要求：

- 使用 db tag
- 使用 json tag

CreateIssueRequest

用于创建 issue。

字段：

字段
Title
Content
Status
Priority

UpdateIssueRequest

用于更新 issue。

字段：

字段
Title
Content
Status
Priority

要求：

- 支持部分更新
- 允许 omitempty

IssueListResponse

用于分页返回。

字段建议：

字段
List
Total

字段

Page

PageSize

八、DAO 层设计

文件：

internal/dao/issue_dao.go

DAO 职责

DAO 只负责：

SQL
数据库交互

DAO 不负责：

JWT
权限判断
HTTP
参数校验

IssueDAO

保存：

db

类型：

*sqlx.DB

NewIssueDAO

作用：

创建 DAO 对象。

参数：

db

返回：

*IssueDAO

CreateIssue

作用：

插入 issue。

参数建议：

Issue

或者：

userID + request

由你决定。

返回：

error

成功：

nil

失败：

数据库错误。

GetIssueByID

作用：

根据 id 查询 issue。

参数：

```
issueID
```

返回：

```
(*Issue, error)
```

规则：

查到

```
return &issue, nil
```

不存在

```
return nil, nil
```

SQL 失败

```
return nil, err
```

ListIssues

作用：

分页查询 issue。

参数至少：

参数

page

pageSize

要求：

使用：

```
LIMIT  
OFFSET
```

返回：

```
([]Issue, total, error)
```

total：

总数据量。

用于前端分页。

UpdateIssue

作用：

更新 issue。

参数建议：

```
issueID  
更新内容
```

返回：

```
error
```

DeleteIssue

作用：

删除 issue。

参数：

issueID

返回：

error

九、Service 层设计

文件：

internal/service/issue_service.go

Service 职责

负责：

业务逻辑
权限控制
参数校验

不负责：

SQL
HTTP

IssueService

保存：

issueDAO

CreateIssue

作用：

创建 issue。

参数：

当前用户ID
CreateIssueRequest

流程：

1. title 不能为空
2. status 必须合法
3. priority 必须合法
4. 调 DAO 创建 issue

user_id:

必须来自：

JWT middleware

不是前端传。

GetIssueByID

作用：

查询 issue 详情。

参数：

issueID

流程：

1. 查询 issue
2. issue 不存在返回错误

返回：

```
(*Issue, error)
```

ListIssues

作用：

分页查询。

参数：

```
page  
pageSize
```

要求：

```
page >= 1  
pageSize 限制最大值
```

返回：

```
IssueListResponse
```

UpdateIssue

本 LAB 最核心函数。

参数：

```
当前用户ID  
issueID  
UpdateIssueRequest
```

流程：

- ```
1. 查询 issue
2. 判断是否存在
3. 判断 issue.user_id == 当前用户ID
4. 校验 status
```

5. 校验 priority
6. 更新 issue

---

## DeleteIssue

流程：

1. 查询 issue
2. 判断是否存在
3. 判断是否属于当前用户
4. 删除

---

## 十、权限控制

本 LAB 最重要部分。

---

### 为什么要有 user\_id

因为：

issue 必须属于某个用户

例如：

issue.user\_id = 创建者

---

### 为什么不能让前端传 user\_id

因为：

前端可以伪造。

正确做法：

JWT  
↓  
middleware  
↓

```
gin.Context
↓
service
```

## 权限判断

更新/删除前必须判断：

```
issue.user_id == 当前登录用户ID
```

否则：

```
403 forbidden
```

## 十一、Controller 设计

文件：

```
internal/controller/issue_controller.go
```

## Controller 职责

负责：

```
HTTP
JSON
Path Param
Query Param
```

不负责：

```
SQL
权限
JWT
```

## CreateIssue

接口:

POST /issues

流程:

1. Bind JSON
2. 从 Context 获取 user\_id
3. 调 service.CreateIssue
4. 返回 JSON

---

## GetIssue

接口:

GET /issues/:id

流程:

1. 获取 path param
2. string -> int64
3. 调 service

---

## ListIssues

接口:

GET /issues?page=1&page\_size=10

流程:

1. 获取 query param
2. 转 int
3. 调 service
4. 返回分页数据

## UpdateIssue

接口：

```
PATCH /issues/:id
```

流程：

1. 获取 issueID
2. 获取 userID
3. Bind JSON
4. 调 service

---

## DeleteIssue

接口：

```
DELETE /issues/:id
```

流程：

1. 获取 issueID
2. 获取 userID
3. 调 service

---

## 十二、Router 设计

文件：

```
internal/router/router.go
```

新增：

```
POST /issues
GET /issues
GET /issues/:id
PATCH /issues/:id
DELETE /issues/:id
```

要求：

全部必须登录。

因此：

```
authGroup
```

下注册。

---

## 十三、分页设计

接口：

```
GET /issues?page=1&page_size=10
```

---

### page

页码。

从 1 开始。

---

### page\_size

每页数量。

建议：

```
最大不超过 100
```

---

### offset 计算

你需要自己计算：

```
offset = (page - 1) * page_size
```

---

## 十四、错误设计

推荐区分：

---

### 400

参数错误。

例如：

非法 status  
非法 priority

---

### 401

未登录。

---

### 403

无权限。

例如：

修改别人的 issue

---

### 404

issue 不存在。

---

## 十五、验收标准

必须通过：

---

### 创建 issue

登录用户可以创建。

---

## 未登录不能创建

返回：

401

---

## 查询 issue 列表

支持分页。

---

## 查询单个 issue

存在返回详情。

不存在返回：

404

---

## 修改自己的 issue

成功。

---

## 修改别人的 issue

返回：

403

---

## 删除自己的 issue

成功。

---

## 删除别人的 issue

返回：

403

## 十六、本 LAB 训练重点

不是 CRUD。

而是：

### 1. 用户与资源关系

user -> issue

### 2. 权限控制

只能修改自己的资源

### 3. RESTful API

POST  
GET  
PATCH  
DELETE

### 4. 分页

LIMIT OFFSET

### 5. Service 业务逻辑

真正开始写：

业务规则